Assignment: RISC vs. CISC — Choosing a Processor

Student Name: Hardik Aggarwal
UID: 25BAI10001
Class: 25BAI601
Subject: Computer Organisation and Architecture

Introduction

A technology company is building two systems, one on RISC and one on CISC, and has to decide which instruction-set design suits a low-power embedded device. In this report I compare the two architectures in a table, work through two small instruction examples, and then give my recommendation for a battery-powered device.

(a) Comparison Table

Parameter	RISC	CISC
Instruction set size	Small, simple instructions	Large, powerful instructions
Instruction length	Fixed, usually 32 bits	Variable, 1 to 15 bytes in x86
Decoding / control unit	Simple, mostly hardwired	Complex, often uses microcode
Addressing modes	Few; only LOAD/STORE use memory	Many; most instructions can use memory
Registers	Large number of registers	Comparatively fewer registers
Cycles per instruction	About 1, good for pipelining	Varies, some need many cycles
Code size	Bigger, more instructions	Smaller, fewer instructions
Power and chip area	Low, fewer transistors	Higher, more complex hardware

Looking at the table, it is clear that the two designs go in opposite directions. RISC keeps the hardware simple and depends on the compiler to build bigger operations out of small instructions. CISC does the opposite and puts the effort into the hardware so that programs come out shorter. When the device runs on a battery, the simpler hardware matters more than the shorter code.

(b) Example Instruction Sequences

Example 1: A = B + C, with A, B and C stored in memory

A CISC machine like the VAX can do this with a single instruction that reads both values from memory and writes the answer back. A RISC machine can only do arithmetic on registers, so the values have to be loaded first and the result stored afterwards. That takes four instructions.

```
CISC:  ADDL3  B, C, A        ; one instruction, memory to memory

RISC:  LW   t0, B            ; load B into a register
       LW   t1, C            ; load C into a register
       ADD  t2, t0, t1       ; add the two registers
       SW   t2, A            ; store the result in A
```

So the CISC code is shorter, 1 instruction against 4. But the RISC instructions are all simple and take about one cycle each, so they flow through the pipeline nicely. The single CISC instruction hides several memory accesses inside it and can take much longer to finish.

Example 2: Copying a block of bytes

Here the difference is even easier to see. On x86 one instruction with a repeat prefix copies the whole block, but on RISC we have to write the loop ourselves.

```
CISC (x86):  REP MOVSB              ; copy CX bytes from [SI] to [DI]

RISC:  loop:  LB    t0, 0(a1)     ; load one byte from source
              SB    t0, 0(a0)     ; store it at destination
              ADDI  a0, a0, 1     ; move destination pointer
              ADDI  a1, a1, 1     ; move source pointer
              ADDI  a2, a2, -1    ; reduce the count
              BNE   a2, zero, loop ; repeat until count is zero
```

The x86 version is much easier to write and read. The RISC loop is longer, but every step in it is basic, so the processor can run it fast and without spending much power.

(c) Processor Recommendation for a Battery-Powered Embedded Device

For this device I would choose a RISC processor, something like an ARM Cortex-M or a RISC-V core. The biggest reason is power. A RISC core has simple decoding and fewer transistors, so there is less switching activity and the battery lasts longer, and battery life is the main requirement here. A simple core also takes less chip area, costs less to make and gives off less heat, which is useful in a small embedded product.

Timing is another point in its favour. Because the instructions are fixed in length and take roughly one cycle each, it is easy to predict how long a piece of code will run. This is important for jobs like reading sensors or controlling a motor, where the response has to arrive on time. The regular format also keeps the pipeline full, so the chip can give decent performance even at a low clock speed, which saves more power. On top of that, most embedded and mobile chips already use RISC cores, so compilers, tools and ready-made chips are easy to get.

CISC would only make sense if code size were the biggest problem, or if the device had to run existing x86 software. Neither applies here, so in my view RISC is the better balance of low power, low cost and enough performance for a battery-powered embedded device.